

CPS_S MATH 483 Final Project: Robot Pencil Grasping and Image Drawing via Jacobian-Based Control

Abdur Islam, Aden Athar

Abstract—We present a robotics system that (1) detects and grasps a pencil from a known pose and (2) draws a user-provided 2-D image on paper by tracking a planned pen-tip trajectory. The core technical contributions are: a forward-kinematics chain based on SE(3) homogeneous transforms, a Jacobian-based inverse-kinematics controller using the Moore–Penrose pseudoinverse, nullspace projection for secondary objectives, and a stroke-decomposition pipeline that converts a bitmap into 3-D end-effector waypoints. We validate the approach in simulation by measuring grasp success rate, RMS pen-tip tracking error, and drawing similarity to the target image. Results confirm that the pseudoinverse controller tracks simple geometric shapes (line, square, circle) with sub-centimetre RMS error, and that nullspace regularisation improves joint-limit avoidance without degrading tracking performance.

I. INTRODUCTION

Robotic manipulation combines perception, planning, and control to let a machine interact with objects in its environment. A particularly informative class of tasks is *contact-rich drawing*: the robot must simultaneously regulate the 3-D position of a tool tip and maintain appropriate contact forces against a surface. Unlike bin-picking, where errors on the order of millimetres are acceptable, drawing requires sub-centimetre accuracy over extended trajectories, making it a useful benchmark for control-loop performance.

Prior art in robot drawing [?], [?] has relied either on offline trajectory optimisation with high-DoF arms or on dedicated end-effector hardware. Our contribution is a *purely algorithmic* pipeline that works with a standard 6-DoF manipulator and off-the-shelf perception, using Jacobian-based control as its sole motion strategy.

Statement of the question. Can we build an algorithmic robotics pipeline that reliably grasps a pencil and reproduces an input image on paper with low tracking error, using forward kinematics, Jacobian pseudoinverse control, and nullspace projection?

The remainder of the paper is organised as follows. Section II surveys related work. Section III formalises the problem. Section IV describes our algorithm. Section V presents experiments and results.

II. RELATED WORK

Robot drawing and calligraphy. Luo et al. [?] use a deep-learning stroke model combined with a 7-DoF arm to replicate brush-stroke calligraphy; their approach requires a large stroke dataset. Berio et al. [?] generate expressive line drawings

by optimising a dynamical-system model of human strokes. Our work avoids learned motion primitives and instead relies entirely on classical differential kinematics.

Jacobian-based control. The Moore–Penrose pseudoinverse solution to the differential IK problem is classical [?]. Nullspace projection for secondary objectives (joint-limit avoidance, singularity robustness) is described in [?]. We apply these standard tools to the drawing task.

Grasping. We use ground-truth pose for the pencil (simulation) and a simple antipodal grasp planner [?]. Pose estimation from depth cameras (e.g. PoseCNN [?]) could replace the ground-truth pose in a hardware deployment.

III. PROBLEM FORMULATION

A. Task

Given a 6-DoF robot arm with a two-finger gripper, a pencil at a known pose, and a flat drawing surface (paper), the robot must:

- 1) Perceive the pencil pose ${}^wT_{\text{pencil}}$ and the paper pose ${}^wT_{\text{paper}}$ in a world frame $\{w\}$.
- 2) Execute a grasp to pick up the pencil without dropping it.
- 3) Convert an input bitmap I into a 2-D drawing trajectory (a sequence of strokes $\mathcal{S}$).
- 4) Track the pen-tip trajectory while alternating pen-down and pen-up contact states.

B. Inputs

- Robot model: kinematic chain of n revolute joints, each described by a DH parameter tuple $(\theta_i, d_i, a_i, \alpha_i)$, plus joint limits $[q_i^{\min}, q_i^{\max}]$.
- Initial joint configuration $q_0 \in \mathbb{R}^n$.
- Perception estimates: pencil pose ${}^wT_{\text{pencil}}$, paper pose ${}^wT_{\text{paper}}$ (simulation ground truth or camera estimate).
- Input image I (bitmap) and drawing parameters: scale s , offset Δp , stroke width w_s .

C. Outputs

- A grasp success indicator (Boolean).
- A time-parameterised joint trajectory $q(t)$ covering grasping and drawing.
- A resulting drawing on paper, evaluated against the target image.

D. Frames and Kinematics

Rigid-body poses are represented as homogeneous transforms in $SE(3)$:

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}, \quad R \in SO(3), p \in \mathbb{R}^3. \quad (1)$$

Forward kinematics maps joint angles to the end-effector pose:

$${}^wT_{ee}(q) = \prod_{i=1}^n {}^{i-1}T_i(q_i). \quad (2)$$

Differential kinematics relates joint velocities to end-effector twist via the manipulator Jacobian $J(q) \in \mathbb{R}^{6 \times n}$:

$$\xi = \begin{bmatrix} \omega \\ v \end{bmatrix} = J(q) \dot{q}. \quad (3)$$

Success criteria.

- *Grasp success rate*: fraction of trials in which the pencil is picked up without dropping.
- *Trajectory tracking error*: $e(t) = \|p_{\text{tip}}(t) - p_{\text{tip}}^*(t)\|$; reported as RMS and peak values.
- *Drawing similarity*: a Chamfer-like edge distance between the target and produced drawing.

IV. ALGORITHM / METHODOLOGY

A. Pipeline Overview

The system is a four-stage modular pipeline:

- 1) **Perception**: obtain ${}^wT_{\text{pencil}}$ and ${}^wT_{\text{paper}}$ from simulation ground truth (or a depth-camera pose estimator for hardware).
- 2) **Grasp planning**: choose a grasp pose ${}^wT_{\text{grasp}}$ relative to the pencil frame using an antipodal heuristic.
- 3) **Trajectory generation**: convert image I to 2-D polylines $\mathcal{S}$, then lift each point to a 3-D waypoint in $\{w\}$ via ${}^wT_{\text{paper}}$.
- 4) **Control**: track desired end-effector twists using Jacobian pseudoinverse control with nullspace projection.

B. Drawing Trajectory Generation

We extract edges from I using the Canny detector and trace them into a set of K polylines:

$$\mathcal{S} = \{\text{polyline}_k\}_{k=1}^K, \quad \text{polyline}_k = \{(x_j, y_j)\}_{j=1}^{m_k}. \quad (4)$$

Each 2-D point is lifted to a 3-D pen-tip location in the paper frame:

$${}^{\text{paper}}p_{\text{tip}} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad z = z_{\text{down}} \text{ (pen-down) or } z_{\text{up}} \text{ (pen-up)}. \quad (5)$$

Finally, waypoints are transformed to the world frame via ${}^wT_{\text{paper}}$.

Algorithm 1: Jacobian pseudoinverse drawing controller

Input: $q_0, {}^wT_{\text{paper}}$, stroke set $\mathcal{S}$
Output: Joint trajectory $q(t)$

```

1 Initialise  $q \leftarrow q_0$ 
2 foreach stroke polyline  $\in \mathcal{S}$  do
3   Move pen to start of polyline with  $z = z_{\text{up}}$ 
4   Lower pen to  $z = z_{\text{down}}$ 
5   foreach waypoint  $p$  in polyline do
6     Compute desired tip pose  ${}^wT_{\text{tip}}^*$ 
7     Compute error  $e$  and reference twist  $\xi_{\text{ref}}$ 
8      $\dot{q} \leftarrow J(q)^+ \xi_{\text{ref}} + (I - J^+ J) \dot{q}_{\text{ref}}$ 
9      $q \leftarrow \text{clip}(q + \dot{q} \Delta t, q^{\min}, q^{\max})$ 
10  Raise pen to  $z = z_{\text{up}}$ 
```

C. Jacobian-Based Tracking Control

At every control cycle with timestep Δt , we compute the desired end-effector twist from position and orientation error:

$$\xi_{\text{ref}} = \begin{bmatrix} k_{\omega} e_{\omega} \\ k_v e_v \end{bmatrix}, \quad (6)$$

where $e_v = p^* - p$ is the position error and e_{ω} is the axis-angle orientation error, with gains $k_v, k_{\omega} > 0$.

The joint velocity command is:

$$\dot{q} = J(q)^+ \xi_{\text{ref}} + (I - J(q)^+ J(q)) \dot{q}_{\text{ref}}, \quad (7)$$

where $J^+ = J^T (J J^T)^{-1}$ is the Moore–Penrose pseudoinverse and $(I - J^+ J) \dot{q}_{\text{ref}}$ is the nullspace term used for secondary objectives (joint-limit avoidance).

Joints are integrated with forward Euler:

$$q \leftarrow \text{clip}(q + \dot{q} \Delta t, q^{\min}, q^{\max}). \quad (8)$$

D. Software Architecture (ROS 2)

For a hardware deployment the pipeline maps onto three ROS 2 nodes:

- `perception_node`: subscribes to an RGBD camera topic and publishes ${}^wT_{\text{pencil}}$ and ${}^wT_{\text{paper}}$.
- `planner_node`: receives poses and the input image; publishes a stamped array of reference twists.
- `controller_node`: subscribes to reference twists, runs Eq. (7) at 100 Hz, and publishes JointTrajectory commands to the hardware driver.

V. EXPERIMENTS / RESULTS

A. Simulation Setup

All experiments are run inside a PyBullet simulation of a 6-DoF UR5 arm. The DH parameters follow the official UR5 specification. A flat paper plane of 20×20 cm is placed at a fixed pose 30 cm in front of the robot base. Control runs at $\Delta t = 10$ ms with gains $k_v = 5 \text{ s}^{-1}$ and $k_{\omega} = 2 \text{ s}^{-1}$.

B. Experiment 1: Grasp Success Rate

The pencil is placed at 50 uniformly-sampled poses inside a 10×10 cm region of the workspace. The gripper attempts the antipodal grasp from a top-down approach direction. Table I summarises the results.

TABLE I
GRASP SUCCESS RATE OVER 50 RANDOMISED PENCIL POSES.

Condition	# successes	Success rate
Nominal (known pose)	50	100%
With ± 5 mm pose noise	16	32%
With ± 10 mm pose noise	9	18%

C. Experiment 2: Trajectory Tracking Error

We command three geometric shapes — a straight line (10 cm), a 10×10 cm square, and a circle with 5 cm radius — and record the pen-tip position error at each timestep. Table II reports RMS and peak errors.

TABLE II
PEN-TIP TRACKING ERROR FOR THREE GEOMETRIC SHAPES.

Shape	RMS error (mm)	Peak error (mm)
Line	1.3	2.0
Square	2.1	3.8
Circle	1.7	3.3

The higher peak error for the square arises at the corners, where the desired twist changes direction abruptly. Adding a small corner-smoothing spline reduces the peak to 4.2 mm.

D. Experiment 3: Drawing Quality

We draw a simple two-stroke line-art image (a house silhouette). The target edge map and the recorded pen path are compared using a directed Hausdorff distance. The mean directed distance is 11.9 mm and the 95th-percentile distance is 21.1 mm.

E. Nullspace Comparison

We compare the controller with and without the nullspace term in Eq. (7). Without the nullspace term, one joint saturates on 9% of drawing steps over a four-lap combined trajectory; with it, saturation drops to 0%.

F. Discussion

The results confirm that Jacobian pseudoinverse control provides sufficient accuracy for line-drawing tasks with a 6-DoF arm. The main failure modes are: (i) singularity proximity causing joint-velocity spikes, mitigated by a damped pseudoinverse $J_{\lambda}^{+} = J^{\top}(JJ^{\top} + \lambda I)^{-1}$; and (ii) cumulative integration drift over long strokes, addressable by periodic re-zeroing against a joint-space IK solution.

REFERENCES